

Vector Databases and Retrieval-Augmented Generation (RAG)

Prof. Surya Prakash

Department of Computer Science & Engineering
Indian Institute of Technology Indore, Indore-453552, INDIA

E-mail: surya@iiti.ac.in

Introduction

- Large Language Models (LLMs) have knowledge cutoffs
- Hallucination is a major challenge
- Need to ground responses in external knowledge
- RAG solves this by combining retrieval + generation

LLMs Have Knowledge Cutoffs

- LLMs are trained on massive but **finite datasets** collected up to a specific point in time, known as the **knowledge cutoff**.
- This means:
 - LLMs **do not have awareness of** events, updates, or documents created after their training period.

LLMs Have Knowledge Cutoffs

- They cannot inherently access:
 - Recent research papers
 - Updated policies, manuals, or databases
 - Organization-specific or private data
- Any response about post-cutoff information is **not guaranteed to be correct**.
- As a result, LLMs are **static knowledge systems**, while real-world information is **dynamic and continuously evolving**.

Hallucination is a Major Challenge

- Hallucination refers to the phenomenon where an LLM generates:
 - Factually incorrect information
 - Fabricated references, citations, or explanations
 - Confident but unsupported answers
- This happens because LLMs:
 - Predict the **most probable next token**, not the **most truthful fact**
 - Lack built-in mechanisms to verify answers against a trusted source
 - Optimize for fluency rather than factual accuracy

Hallucination is a Major Challenge

- In high-stakes applications such as following, hallucinations can lead to **serious consequences**, reducing trust in AI systems.
 - Healthcare
 - Legal systems
 - Academic research
 - Customer support

Need to Ground Responses in External Knowledge

- Grounding means constraining LLM outputs to **verifiable, external sources** rather than relying solely on internal model knowledge.
- Grounded systems:
 - Retrieve relevant documents from trusted repositories
 - Provide context to the LLM before generation
 - Ensure responses are **evidence-based and traceable**

Need to Ground Responses in External Knowledge

- Benefits of grounding:
 - Reduces hallucinations
 - Improves factual correctness
 - Enables domain-specific and up-to-date responses
 - Allows system updates **without retraining the model**
- This motivation directly leads to **Retrieval-Augmented Generation (RAG)**, where LLMs act as **reasoning engines** rather than knowledge stores.

Retrieval-Augmented Generation (RAG)

- RAG is a hybrid AI framework that **enhances LLMs** by dynamically retrieving relevant information from external knowledge sources and using it to **ground the generated responses**.
- Instead of relying solely on the model's internal parameters, RAG allows the model to **consult external documents at inference time**.

Summary – Why RAG is Needed?

- LLMs have **fixed knowledge cutoffs**
- High risk of **hallucinated responses**
- No access to private or updated data
- Retraining LLMs is expensive and slow
- RAG addresses these limitations by **decoupling knowledge storage from reasoning**.

Core Components of RAG

- **Document Source**
 - PDFs, manuals, FAQs, research papers, databases
- **Text Chunker**
 - Splits documents into manageable, semantically meaningful chunks
- **Embedding Model**
 - Converts text chunks and queries into vector representations
- **Vector Database**
 - Stores embeddings (e.g., FAISS, Chroma)
 - Enables semantic similarity search
- **Retriever**
 - Fetches top-k relevant chunks for a given query
- **Generator (LLM)**
 - Generates answers using retrieved context only

Document Source

- The document source refers to the **external knowledge repository** from which information is retrieved to support and ground the responses generated by a LLM in a RAG system.
- These documents act as the **authoritative knowledge base** for the system.
- Example
 - PDF: Research papers, reports, policy documents, Conference guidelines, thesis regulations
 - Manuals: Software user manuals, equipment operating procedures
 - FAQs: Admission FAQs, customer support FAQs
 - Databases: Structured or semi-structured data

Document Source

- **Document Source is Not Updated (Static)**
- **What problem exists?**
- LLM may:
 - Hallucinate facts
 - Rely only on its pretrained knowledge
- Documents may be:
 - Old policies
 - Archived reports
 - Fixed manuals

How RAG Still Helps

→ **Grounded Answers (Most Important)**

- LLM answers **only using retrieved documents**
- Even if documents are old:
 - Answers are **consistent**
 - **Traceable to source**
 - Less hallucination

→ Better to give a *correct answer based on old data* than a *confident hallucination*

Page Ranking Vs. RAG

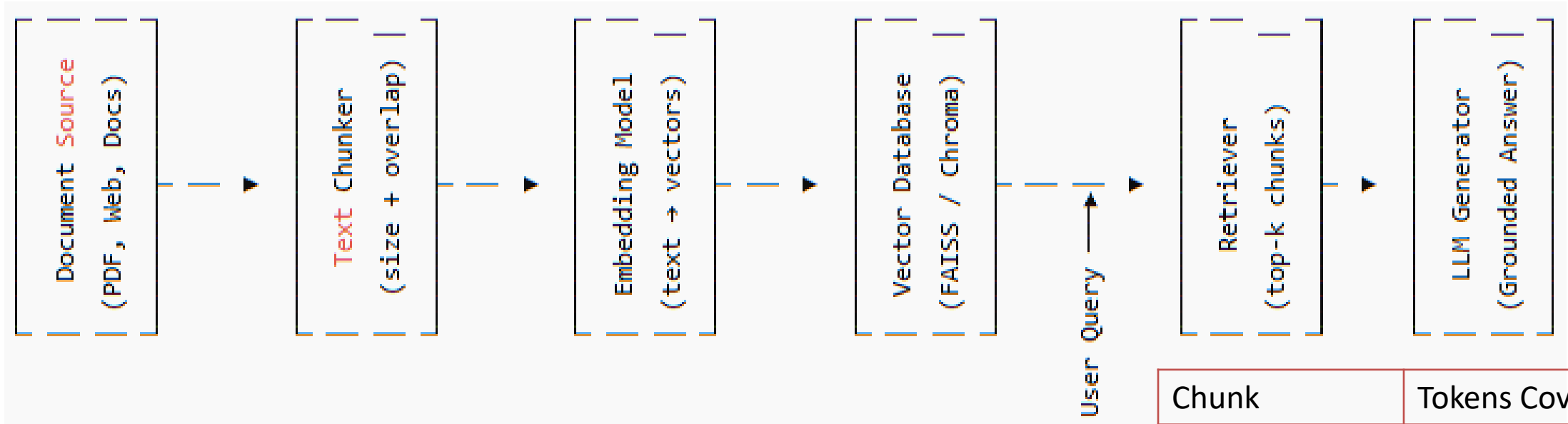
How Google works	How RAG works
Offline (preprocessing) - Crawls the web - Indexes pages - Computes PageRank (importance) - Stores keyword indexes Online (when you search) - Matches keywords - Applies ranking signals - Returns ranked pages ➔ Ranking is page-level and heavily precomputed	Offline - Documents are chunked - Each chunk → embedding - Stored in vector DB ➔ No ranking is done yet Online (query time) - Query → embedding - Similarity computed on the fly - Chunks ranked by vector similarity ➔ Ranking is dynamic and query-dependent

Text Chunker

- A **Text Chunker** is a preprocessing component applied on the **Document Source**
- It splits large documents into smaller, meaningful text segments (**chunks**) so they can be efficiently embedded, stored, and retrieved in a vector database in RAG systems.
 - Smaller chunks (e.g., 200–1,000 tokens)
- Chunking is essential because LLMs have **context length limits**, and retrieval works best on **focused, semantically coherent text**.

Text Chunker

- Text chunking is applied to the document source before embedding, so that smaller, semantically meaningful chunks can be stored and retrieved efficiently from the vector database.



Tokens: T1 T2 T3 ... T100 T101 T102 ... T200

Chunking with: Size = 100 tokens, Overlap = 20 tokens

Size + Overlap

Chunk	Tokens Covered
Chunk 1	T1 – T100
Chunk 2	T81 – T180
Chunk 3	T161 – T260

Text Chunker

- Chunks can be generated based on the number of words, but in practice they are *usually* generated based on **tokens**, not words.
- Example: “ChatGPT is powerful!”

Item	Count
Words	3
Tokens	4–5 (model-dependent) Possible tokens: ["Chat", "GPT", "is", "power", "ful", "!"]

Example - Text Chunker

- Chunking a University Admission PDF
- Original Text (Excerpt from PDF)
 - “Applicants must hold a Master’s degree with at least 60% marks. Candidates belonging to reserved categories are eligible with 55%. Admission is based on an entrance examination followed by an interview. The application deadline is 31st May.”

Generated Chunks

- **Chunk 1 (Eligibility):**
 - “Applicants must hold a Master’s degree with at least 60% marks. Candidates belonging to reserved categories are eligible with 55%.”
- **Chunk 2 (Selection Process):**
 - “Admission is based on an entrance examination followed by an interview.”
- **Chunk 3 (Deadline):**
 - “The application deadline is 31st May.”

How Chunking Helps in Retrieval

- **User Query:**
 - *“What is the eligibility for PhD admission?”*
- → Only **Chunk 1** is retrieved
- → LLM generates a precise, grounded answer
- → No irrelevant deadline or process text included

Common Chunking Strategies

Strategy	Description	Use Case
Fixed-size	Split by tokens/words	Simple documents
Overlapping	Chunks share context	Long narratives
Semantic	Split by meaning	Research papers
Section-based	Headings & sections	Manuals, PDFs

Embedding Model

- An Embedding Model converts text (sentences, paragraphs, or documents) into high-dimensional numerical vectors such that semantically similar texts are close in vector space, enabling effective retrieval in RAG systems.
- Texts with similar meaning produce vectors that are close to each other in vector space.
- Why Embeddings are Important
 - Enable semantic search instead of keyword search
 - Allow similarity comparison using distance metrics
 - Form the backbone of vector databases and RAG

Embedding Model

- **How RAG Locates a Document**
- **Documents are preprocessed (offline step)**
 - Documents → **chunked**
 - Each chunk → converted to a **vector embedding**
 - Stored in a **vector database** (FAISS / Chroma)
- **Steps:**
 - Document → Chunks → Embeddings → Vector DB

Example: Embedding University Admission Text

Text Chunks

- **Chunk A:**
 - “Applicants must have a Master’s degree with at least 60% marks.”
- **Chunk B:**
 - “Candidates need a postgraduate degree with minimum 60 percent score.”
- **Chunk C:**
 - “The application deadline is 31st May.”

Example: Embedding University Admission Text

- **Embedding Generation**

- Each chunk is passed to an embedding model (e.g., Sentence-BERT, OpenAI embeddings):

Chunk A → [0.12, -0.45, 0.88, ... , 0.31]
Chunk B → [0.10, -0.47, 0.86, ... , 0.29]
Chunk C → [-0.72, 0.15, -0.11, ... , 0.90]

- **Semantic Similarity Observation**

- **Chunk A and Chunk B** → vectors are **close**
- **Chunk C** → vector is far away
- This shows embeddings capture **meaning**, not exact wording.

Query Embedding Example

- **User Query:**
 - *“What is the minimum qualification required?”*
- Query is embedded as: `Query` → `[0.11, -0.46, 0.87, ... , 0.30]`
- This vector is closest to **Chunk A** and **Chunk B**.
- **Result in RAG**
 - Vector database retrieves **Chunk A / B**
 - These chunks are passed to the LLM
 - LLM generates a **grounded answer**

Vector Database

- A **Vector Database** is a specialized database designed to **store, index, and search high-dimensional vector embeddings** efficiently, enabling **semantic similarity search** rather than exact keyword matching.
- A vector database stores and searches embeddings using similarity metrics to retrieve semantically relevant information for RAG systems.
- In RAG systems, it acts as the **retrieval engine** that supplies relevant context to the LLM.

Role of Vector Database in RAG

- Stores embeddings of document chunks
- Stores embedding of user queries
- Retrieves **top-k semantically similar chunks**
- Provides grounded context for response generation

Example: University FAQ Knowledge Base

- Stored Chunks and Embeddings
 - Each text chunk is converted into a **vector embedding** and stored.

Chunk	Text	Vector
C1	“Master’s degree with 60% required”	v_1
C2	“Entrance exam and interview process”	v_2
C3	“Application deadline is 31st May”	v_3

Query Example

- **User Query:**
 - *“What is the eligibility for PhD admission?”*
- Query is embedded $\rightarrow \mathbf{v}_q$
- Vector database compares $\mathbf{v}_q$ with $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$
- $\mathbf{v}_1$ has highest similarity
- **Retrieved Result**
 - *“Applicants must hold a Master’s degree with at least 60% marks.”*
- This chunk is sent to the LLM as **grounding context**.

Similarity Search

- Vector databases use distance metrics such as:
 - Cosine similarity
 - Euclidean distance
 - Inner product
- Nearest vectors represent **closest meaning**.

Indexing for Efficiency

- To handle large-scale data, vector DBs use:
 - Flat indexing (exact search)
 - IVF (Inverted File Index)
 - HNSW (Hierarchical Navigable Small World graphs)

Popular Vector Databases

- **FAISS**
 - High-performance similarity search
- **Chroma**
 - Lightweight, LLM-friendly
- **Pinecone**
 - Managed service
- **Milvus, Weaviate**
 - Scalable open-source options

Vector DB *vs* Traditional DB

Aspect	Traditional DB	Vector DB
Search type	Keyword/exact	Semantic
Data type	Text, numbers	High-dim vectors
Query	SQL	Similarity search
Use in RAG	Not suitable	Core component

Retriever

- A **Retriever** is the component of a RAG system that **fetches the most relevant document chunks** from a vector database based on the **semantic similarity** between the user query and stored embeddings.
- The retriever selects the most semantically relevant document chunks from a vector database to ground LLM responses in RAG systems.
- It acts as the **bridge between the vector database and the LLM**.
- **Role of the Retriever**
 - Receives the embedded user query
 - Performs similarity search in the vector database
 - Selects **top-k relevant chunks**

Example: University FAQ Bot

- **Stored Chunks**

- “Applicants must hold a Master’s degree with at least 60% marks.”
- “Admission is based on an entrance examination and interview.”
- “The application deadline is 31st May.”

- **User Query**

- “*What is the eligibility for PhD admission?*”

Example: University FAQ Bot

- Retriever Operation
 - Query embedding is generated
 - Retriever computes similarity with stored vectors
 - Top-1 or Top-2 chunks selected
- Retrieved Chunk:
 - “Applicants must hold a Master’s degree with at least 60% marks.”

Types of Retrievers

- **1. Dense Retriever**
 - Uses embeddings and vector similarity
 - Captures semantic meaning
 - Most common in RAG
- **2. Sparse Retriever**
 - Keyword-based (e.g., BM25)
 - Exact term matching
- **3. Hybrid Retriever**
 - Combines dense + sparse retrieval
 - Improves recall and precision

Retriever Parameters

- **Top-k:**
 - Number of chunks retrieved
- **Similarity threshold:**
 - Minimum relevance score
- **Filtering:**
 - Metadata-based filtering (date, source, section)

Retriever *vs* Vector Database

Component	Purpose
Vector Database	Stores and indexes embeddings
Retriever	Queries the DB and selects relevant chunks

- **Impact on RAG Performance**
 - Poor retriever → irrelevant context
 - Irrelevant context → hallucinated answers
 - Strong retriever → accurate, grounded generation

Generator

- The **Generator** is the Large Language Model (LLM) component in a RAG system that **produces the final natural language response** by combining the **user query** with the **retrieved, grounded context** supplied by the retriever.
- It performs **reasoning and language generation**, not knowledge storage.

Role of the Generator

- Receives:
 - User query
 - Retrieved document chunks
- Synthesizes information from context
- Generates a **coherent, grounded answer**
- Avoids relying on internal model memory

Example: University FAQ Bot

- Input to the Generator
- Retrieved Context:
 - “Applicants must hold a Master’s degree with at least 60% marks.”
- User Query:
 - “What is the eligibility for PhD admission?”
- **Generator Output**
 - *“To be eligible for PhD admission, candidates must have a Master’s degree with at least 60% marks.”*
 - Answer is concise
 - Fully grounded in retrieved text
 - No hallucination

Prompt Construction for the Generator

- A typical RAG prompt:
- **Instruction:**
Answer the question using only the provided context.
- **Context:**
[Retrieved document chunks]
- **Question:**
[User query]
- This prompt **constrains generation** to retrieved knowledge.

Generator Models

- Generator Models
 - GPT-family models
 - LLaMA
 - Mistral
 - T5 / FLAN-T5

Generator *vs* Retriever

Component	Function
Retriever	Finds relevant information
Generator	Produces final answer

Common Generator Issues

- Hallucination if context is weak
- Overly verbose answers
- Ignoring provided context

Mitigation:

- Strong prompts
- Context-only constraints
- Answer length control

RAG Workflow

- User submits a query
- Query is converted into an embedding
- Vector database retrieves relevant document chunks
- Retrieved chunks are appended to the prompt
- LLM generates a grounded response

Example: FAQ Bot for a University Website

(RAG Workflow)

- **Step 1:**
 - **User Submits a Query**
- **User Question:**
 - *“What is the eligibility criteria for the PhD program?”*

Example: FAQ Bot for a University Website

(RAG Workflow)

- **Step 2: Query is Converted into an Embedding**
 - The question text is passed to an **embedding model**
 - It is converted into a **high-dimensional vector**
 - This vector captures the **semantic meaning** of the query
- *Example:*
 - “*eligibility*”, “*PhD*”, “*criteria*” are semantically encoded

Example: FAQ Bot for a University Website

(RAG Workflow)

- **Step 3: Vector Database Retrieves Relevant Document Chunks**
 - The vector database (e.g., **Chroma** / **FAISS**) searches stored embeddings
 - It finds document chunks such as examples below:
- **Example** of Retrieved Chunks:
 - *“Applicants must have a Master’s degree with at least 60% marks...”*
 - *“Candidates must qualify the university entrance examination...”*
- These chunks are **semantically closest** to the query embedding.

Example: FAQ Bot for a University Website

(RAG Workflow)

- **Step 4: Retrieved Chunks are Appended to the Prompt**
 - The system constructs a prompt like below
- **Context:**
 - “Applicants must have a Master’s degree with at least 60% marks. Candidates must qualify the university entrance examination.”
- **Question:**
 - “What is the eligibility criteria for the PhD program?”
- This ensures the LLM answers **only using the provided context.**

Example: FAQ Bot for a University Website

(RAG Workflow)

- Step 5: LLM Generates a Grounded Response
- LLM Output:
 - *“To be eligible for the PhD program, candidates must hold a Master’s degree with a minimum of 60% marks and must qualify the university entrance examination.”*
- ✓ The answer is **fact-based**
 - ✓ No hallucination
 - ✓ Fully grounded in retrieved documents

Example: FAQ Bot for a University Website

(RAG Workflow)

- Why This is RAG
 - Knowledge is retrieved **before** generation
 - LLM does not rely on its internal memory
 - Answers can be traced back to source documents

RAG *vs* Traditional LLMs

Aspect	Traditional LLM	RAG
Knowledge source	Model weights	External documents
Updates	Retraining needed	Instant document updates
Hallucination risk	High	Significantly reduced
Domain specificity	Limited	Strong

Applications of RAG

- FAQ and customer support bots
- Academic and research assistants
- Legal and policy analysis systems
- Enterprise knowledge search
- Healthcare information systems

Challenges in RAG

- Poor chunking reduces retrieval quality
- Irrelevant context injection
- Latency due to retrieval
- Evaluation of grounding and faithfulness

RAG SUMMARY

RAG Summary

- Retrieval-Augmented Generation is a framework that improves LLM reliability by retrieving relevant external information and grounding generated responses in that retrieved context.
- **RAG is not a model itself.**
 - It's a **framework** to combine an LLM with external knowledge (documents) to produce grounded answers.
 - Any capable LLM can be used with RAG.

Vector Database

- A database optimized for **vector embeddings**
- Stores high-dimensional numerical representations
- Supports **similarity search** (nearest neighbor search)
- Core operation:
 - **semantic retrieval**

Why Vector Databases are Needed

- Traditional DBs → exact match (SQL, keywords)
- Vector DBs → **semantic match**
- Enables:
 - Document search
 - Question answering
 - Recommendation systems
 - RAG pipelines

Embeddings

- **Embeddings → The Foundation**
- Text → numerical vectors using embedding models
- Similar meaning → closer vectors
- Popular models:
 - OpenAI embeddings
 - Sentence-BERT
 - Instructor / E5 models

Similarity Metrics

- Cosine similarity (most common)
- Euclidean distance
- Inner product
- Choice impacts retrieval accuracy

Popular Vector Databases

- FAISS (Facebook AI Similarity Search)
- Chroma
- Pinecone
- Weaviate
- Milvus

- *Focus of this lecture:*
 - *FAISS & Chroma*

FAISS Overview

- Open-source library by Meta
- Optimized for **fast similarity search**
- Supports:
 - CPU & GPU
 - Flat, IVF, HNSW indexes
- Used in large-scale research systems

Chroma Overview

- Lightweight, developer-friendly vector DB
- Designed for **LLM applications**
- Easy Python integration
- Persistent storage + metadata support
- Ideal for **RAG & FAQ bots**

FAISS *vs* Chroma

Feature	FAISS	Chroma
Ease of use	Medium	Very Easy
Persistence (storing the vector database to disk so you don't have to recompute embeddings every time.)	Manual (FAISS is primarily in-memory , but you can save and load indices)	Built-in
Metadata filtering	Limited	Strong
RAG suitability	High	Very High

- Without persistence → embeddings exist only in memory → lost when session ends
- With persistence → embeddings are saved to **disk** → can be loaded later

Text Chunking: Why Needed

- LLMs have **context length limits**
- Large documents must be split
- Chunking improves:
 - Retrieval precision
 - Answer grounding
- Each chunk → separate embedding

Thank You